

Introduction to R for Data Science

INF2167, Week 1

Inessa De Angelis

May 7, 2026

Meet Inessa

- PhD Candidate at the Faculty of Information
- Master of Information from the University of Toronto and Honours Bachelor of Arts from the University of Ottawa
- Over 10 years of experience working in federal Canadian politics

My Research

- My SSHRC-funded dissertation focuses on developing new computational and statistical methods to better measure the forms and impact of online harassment faced by Canadian politicians
- Also co-authoring a book about televised leaders' debates in Canada, under revision and to be published by *University of Toronto Press*
- More broadly, my research focuses on political communication, digital and social media, gender and political representation, computational social science, and data science

Course Objectives

- We will learn to use the statistical programming language, R to do data science to answer social science research questions
- This course focuses on providing students with skills to approach various research questions that involve analysis of social science data, including collection, cleaning, visualization, descriptive statistics, regression, and classification
- We will simultaneously think critically about the socio-political, economic, and ethical questions underpinning data, learn how to undertake reproducible workflows, and clearly communicate our results

Course Plan

- Week 1: May 7th - Introduction to the course and R
- Week 2: May 14th - Reproducible workflows and GitHub
- Week 3: May 21st - Data wrangling and visualization
- Week 4: May 28th - Data sources, collection, and sampling
- Week 5: June 4th - In-class test 1
- Week 6: June 11th - Data sources, exploratory data analysis and descriptive statistics

Course Plan (continued)

- June 18th: **No class**, final project proposal due by 11:59pm on Quercus
- June 25th: **No class**, reading week
- Week 7: July 2nd – Modeling 1 – Linear Regression
- Week 8: July 9th – Modeling 2 – Logistic Regression
- Week 9: July 16th – In-class test 2
- Week 10: July 23rd – Introduction to machine learning
- Week 11: July 30th – Course review
- Week 12: August 6th – Project presentations and course wrap up

Required textbooks

All required textbooks are available online for free or a hard copy can be purchased:

- Alexander, R. (2023). *Telling Stories with Data: With Applications in R*. Chapman and Hall/CRC.
- Gelman, A., Hill, J., & Vehtari, A. (2021). *Regression and Other Stories*. Cambridge University Press.
- Ismay, C., Kim, A. Y., & Valdivia, A. (2025). *Statistical Inference via Data Science: A Modern Dive into R and the Tidyverse* (2nd ed.). Chapman and Hall/CRC.

Computing

- Throughout the course, we will use `R` in all examples. In particular, you will learn and be expected to code in the `tidyverse` style
- All in-class participation activities and final project proposal and paper will be written using Quarto
- You will need to have R and RStudio installed on your computer and have a GitHub account

Quarto

- Quarto documents combine natural language and code (including R and Python) in one file, so you can discuss your ideas and show your results together
- It is the successor to R Markdown
- They are “live” documents, meaning that when we “knit” or render them, the code executes and automatically generates outputs such as tables and figures directly within the document
- You can export to many formats (HTML, PDF, Word, presentations), making it easy to share reproducible documents and slides

Evaluation

- In-class participation and activities (15%)
- 2 In-class tests (40%; 20% each)
- Final project proposal (10%)
- Final project in-class presentation (10%)
- Final project paper (25%)

In-class participation and activities (15%)

- Regular attendance and active participation are essential for both your learning and the overall success of this course
- Everyone begins with a default participation grade of 10/15. Actively participating and submitting the in-class lab assignments throughout the course will raise your grade, while missing classes and not actively participating will lower it
- The in-class lab assignments will be graded on a pass/no pass basis

In-class tests (40%)

- There will be two closed-book tests on paper during the semester, during Week 5 (June 4th) and Week 9 (July 16th)
- Each test will include a combination of multiple-choice questions and short written responses
- Questions will be based on material covered in the course readings, weekly lectures, and in-class activities
- Expect questions on broader social science research design and data science, as well as more specific technical questions about R packages and functions

Final project proposal (10%)

- Working in your pairs I will assign during week 2, you will create a proposal for your final project, which explores a research question using methods from class
- Your proposal should outline your research question(s) and hypotheses, some relevant literature, your dataset and methods, and exploratory data analysis
- Your proposal should be 3–4 pages long and be submitted via Quercus by June 18th at 11:59pm
- A rubric and more information will be posted on Quercus ahead of the deadline

Final project in-class presentation (10%)

- In the last week of class (August 6th), both members of each group will present their project
- Each presentation will be approximately 5-8 minutes long and must include presentation slides
- Presentations should generally include: introduction/motivation, data, methods, findings (or expected findings), and a brief summary
- A rubric and more information will be posted on Quercus ahead of the deadline

Final project paper (25%)

- Working in your assigned pairs, you will submit a final project paper that presents your analysis of a research question using the methods from this course
- Your paper should include a short abstract, introduction, relevant literature, dataset and methods, results and findings, and discussion and conclusion
- Your paper should be 10–12 pages long and submitted via Quercus by August 13th at 11:59pm.
- A rubric and more information will be posted on Quercus ahead of the deadline

General Course Policies

- I will respond to emails within 48 hours (excluding evenings and weekends). Please write the course code (INF2167) in the subject line
- You can address me as **Inessa** (pronounced *ee-NES-sah*)
- We will use both Quercus and GitHub as official class pages
 - All in-class labs and assignments must be submitted through Quercus. Quercus will also be used for announcements and posting grades
 - The class GitHub repository, <https://github.com/inessadeangelis/INF2167>, will be used for in-class examples, live coding, and distributing in-class labs and participation activities

Late Assignments, Grading and Appeals

- Late assignments will be penalized 5% per day, including weekends and holidays. Assignments more than 7 days late will not be accepted or graded
- Re-grading or appeals: I will not discuss your assignment for at least 48 hours. If you want anything re-graded, you must prepare a single page memo outlining your case. This memo needs to be serious and specific. Note your grade may go up, DOWN, or remain the same
- Accommodation requests need to be made using the [course notifications/accommodations form](#) before the due date/class

Course Policy on AI

- **R code:** If you use AI for the development and/or de-bugging of code, the use of such tools **MUST** be explicitly acknowledged and cited in submitted work. All relevant prompts and responses should be included in a text file in your repo and thoroughly commented in your scripts. Failure to do so may result in penalties
- **Written work:** The use of generative AI tools or apps for written assignments in this course, including the final project proposal and paper is **PROHIBITED**. Representing one's own idea, or expression of an idea that was AI-generated may be considered an academic offense in this course. You may not copy or paraphrase from any generative AI applications, for the purpose of completing written course assignments

Academic Integrity and Plagiarism

- Academic offenses include (but are not limited to): plagiarism, cheating, copying R code, communication/extra resources during closed book assessments, submitting the same work twice, falsifying institutional documents, and purchasing labor for assessments
- Sources of R code must be acknowledged with a proper citation for extensive use and commented immediately next to the code if a few lines are used
- If you have questions or concerns about what constitutes appropriate academic behaviour or research and citation methods, please email me

Questions?

- Come to office hours on Zoom, Mondays from 12:00-1:00pm (link and passcode on the syllabus)
- Or email me for an appointment outside of office hours (Inessa.DeAngelis@mail.utoronto.ca)

Survey on Quercus

- Go to the “Quizzes” section on Quercus
- Please fill out the survey to help me learn about you so that I can more effectively teach this course

What is data science?

- There is no agreed definition of data science (Alexander, 2023)
- Wickham et al. (2023) say it “... allows you to turn raw data into understanding, insight, and knowledge”
- Timbers et al. (2022) define it as “the process of generating insight from data through reproducible and auditable processes”
- In essence, we are interested in extracting insights and knowledge from data using systematic and reproducible methods

Applying data science to social science

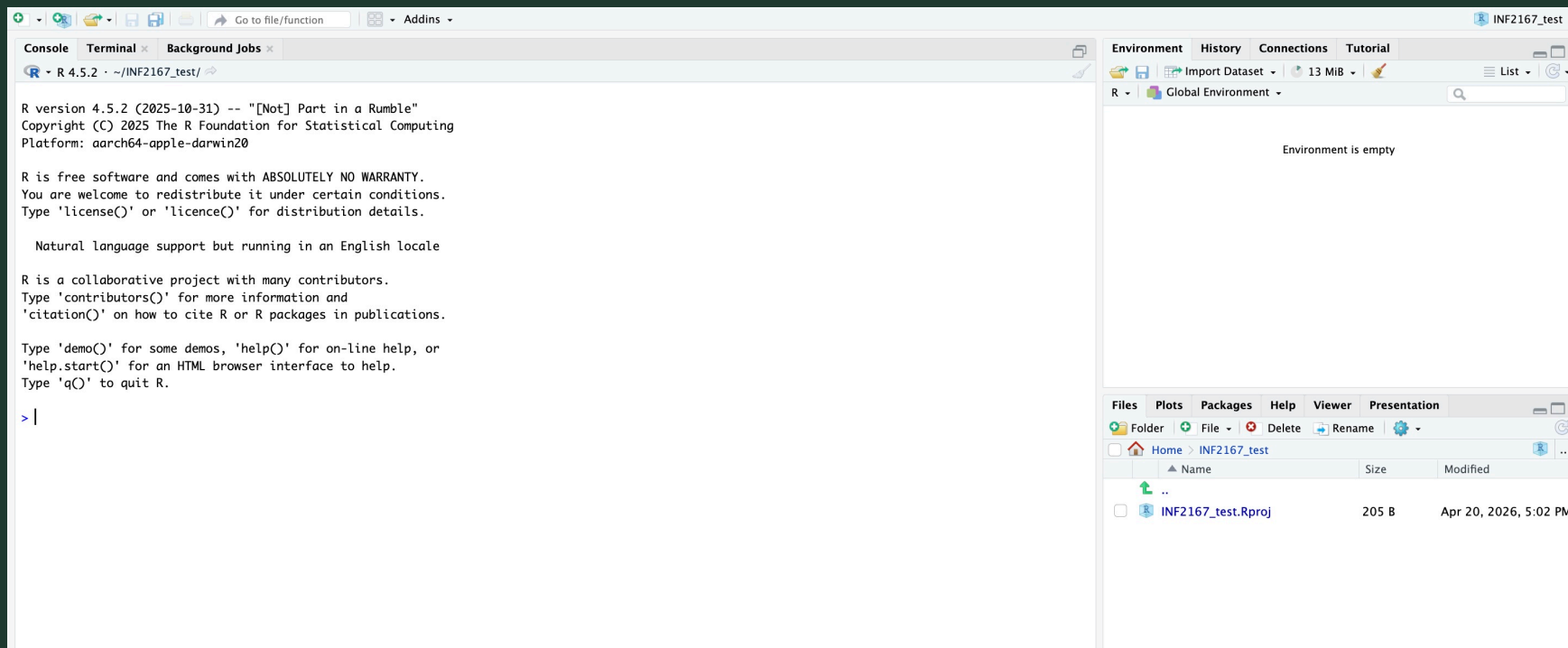
- Means using computational and statistical methods to analyze social data in order to understand human behaviour, social structures, and societal change
- Combines theories from the social sciences with computational and statistical tools (for example, regression, machine learning, network analysis, text analysis)
- Working with social data (for example, surveys, administrative records, social media data, policy documents)
- Aims to generate insights about society in a systematic and reproducible way

Introduction to R & R Studio

- R is an open-source programming language used for statistical computing and data analysis
- RStudio is an integrated development environment (IDE) that provides an interface by adding many convenient features and tools
- We'll always work in RStudio and not in the R application
- Unlike Excel, SPSS, or other statistical software programs, R is an interpreted language, meaning you need to need to write code, not click buttons for functions to work

R Studio

- When we open RStudio it will look like this:
- There are three panels dividing the screen: the *console* pane, the *files* pane, and the *environment* pane
- You can learn more in the [RStudio IDE Cheatsheet](#)



Basic programming concepts and terminology

- **Console pane:** where you enter in commands
- **Running code:** the act of telling R to perform an act by giving it commands in the console
- **Objects:** where values are saved in R
- **Data types:** integers, doubles/numerics, logicals, and characters
- **Data frames:** rectangular spreadsheets with rows (observations) and columns (variables)

Basic programming concepts and terminology (continued)

- **Functions**, also called **commands**: Functions perform tasks in R
- Functions take in inputs called arguments and return outputs
- You can either manually specify a function's arguments or use the function's default values
- Examples: `read_csv()`, `mutate()` and `group_by()`

Base R

- “**Base R**” refers to the core, default functionality that comes installed with R
- It provides the fundamental functions for data manipulation, statistics, and graphics without requiring additional packages
- In this class, we focus on the **tidyverse** because its consistent design makes data science easier to learn, while still building on base R functions
- You can read more about base R in *R for Data Science*, **Chapter 27**

Installing and loading packages

- R packages extend the functionality of R by providing additional functions, data, and documentation
- To install a package on your computer (you will only need to do this once) we use `install.packages()`
 - For example, `install.packages("tidyverse")`
- When we want to load and use the package, we use `library()`
 - For example, `library(tidyverse)`

Political Representation and Toronto City Councillor Meeting Attendance

- Political representation is the mechanism by which political actors (representatives) act on behalf of citizens (constituents) in policymaking, essentially “making present” the voices and interests of the people
- This is a key feature of representative democracies
- Theorist Hanna Pitkin (1967) defines four types of political representation: descriptive, substantive, symbolic, and formalistic

Political Representation and Toronto City Councillor Meeting Attendance

- Attendance refers to physically showing up and participating in the representative processes
- Dovi (2020, p. 562) says that attendance resembles *descriptive representation* – “the physical inclusion within representative processes of representatives who ‘look like’ or ‘share certain experiences with’ those they represent”
- Representatives must be physically present in some sense for representation to occur

Example: Toronto City Councillor Meeting Attendance Records

- The City of Toronto provides open data about civic decision and policy-making. This includes data on municipal elections, daycares, shelters, and even raccoon activity
- Going back to Pitkin (1967) and Dovi's (2020) research, analyzing City of Toronto councillors' attendance records, which are available from Open Data Toronto, can tell us about how they undertake their representative duties
- An example research question for this could be: do councillors who have higher attendance at council and committee meetings “better” represent their constituents?

Example (continued)

Let's load our R packages:

```
1 library(tidyverse)
2 library(opendatatoronto)
```

- `tidyverse` is a collection of R packages that make it easier to import, clean, manipulate, and visualize data (Wickham et al., 2019)
- `opendatatoronto` allows us to access datasets from the City of Toronto's open data portal and downloads them directly in R (Gelfand, 2022)

Example (continued)

Now, lets find and download our councillor attendance dataset from Open Data Toronto:

```
1 data <- list_package_resources("9426c94d-abdb-45fe-9a81-79be40d204d2") |>  
2   filter(str_detect(id, "4f8317bd-566b-4e15-b3b7-59a4a398a856")) |>  
3   get_resource()
```

- We start by assigning the dataset we want to retrieve from Open Data Toronto to an object called `data`
- The `<-` is the assignment operator. Anything on the right-hand side of `<-` is assigned to the object `data`

Example (continued)

```
1 data <- list_package_resources("9426c94d-abdb-45fe-9a81-79be40d204d2") |>
2   filter(str_detect(id, "4f8317bd-566b-4e15-b3b7-59a4a398a856")) |>
3   get_resource()
```

- `list_package_resources` is used to retrieve a list of all available resources (files) associated with a specific dataset package (9426c94d-abdb-45fe-9a81-79be40d204d2) on the Open Data Toronto portal
- “`|>`” in R is called the pipe operator and it’s typically read as “then” or “and then”
- It allows us to link together multiple functions, passing the result of one function as the input to the next. You might see the pipe operator styled as “`%>%`” in older code

Example (continued)

- We use the `filter` and the `str_detect` functions from `tidyverse` to subset rows in a dataset based on whether a specific pattern is found within a given column

```
1 data <- list_package_resources("9426c94d-abdb-45fe-9a81-79be40d204d2") |>  
2   filter(str_detect(id, "4f8317bd-566b-4e15-b3b7-59a4a398a856")) |>  
3   get_resource()
```

- We then use the `get_resource()` function to access and download the dataset we want based on the dataset's metadata (such as its URL or ID)

Example (continued)

- We can save our councillor attendance dataset as a CSV file

```
1 write_csv(x = data, file = "councillor_data.csv")
```

- **Note:** always use `write_csv` (from `readr`) **instead of** `write.csv` (from base R)
- Using `write.csv` can cause issues in your dataset, such as adding extra columns or changing data types

Example (continued)

- Now, we can load our recently saved councillor attendance dataset using `read_csv`

```
1 data <- read_csv("councillor_data.csv")
```

- **Note:** always use `read_csv` (from `readr`) **instead of** `read.csv` (from base R)

Example (continued)

Lets get an overview of the entire dataset using the `glimpse()` function:

```
Rows: 12,137
Columns: 10
$ `_id`          <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14...
$ Term           <chr> "2022-2026", "2022-2026", "2022-2026", "2022-...
$ `First Name`   <chr> "Paula", "Paul", "Brad", "Alejandra", "Jon", ...
$ `Last Name`    <chr> "Fletcher", "Ainslie", "Bradford", "Bravo", "...
$ Committee      <chr> "CreateT0", "City Council", "City Council", "...
$ `MTG #`        <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
$ `Session Date` <date> 2022-11-18, 2022-11-23, 2022-11-23, 2022-11-...
$ `Session Type` <chr> "Morning", "Afternoon", "Afternoon", "Afterno...
$ `Session Start-End Time` <chr> "09:31AM - 09:50AM", "14:03PM - 15:25PM", "14...
$ Present        <chr> "N", "Y", "Y", "Y", "Y", "Y", "Y", "Y", "Y", ...
```

- We can see that there are 12,137 rows (*observations*) and 10 columns (*variables*)

Example (continued)

If we want to see just the first 6 rows of our dataset, we can use the `head()` function:

```
# A tibble: 6 × 10
  `_id` Term      `First Name` `Last Name` Committee    `MTG #` `Session Date`
  <dbl> <chr>      <chr>        <chr>        <chr>        <dbl> <date>
1     1 2022-2026 Paula        Fletcher    CreateT0         1 2022-11-18
2     2 2022-2026 Paul          Ainslie     City Council      1 2022-11-23
3     3 2022-2026 Brad          Bradford    City Council      1 2022-11-23
4     4 2022-2026 Alejandra     Bravo       City Council      1 2022-11-23
5     5 2022-2026 Jon           Burnside    City Council      1 2022-11-23
6     6 2022-2026 Shelley      Carroll     City Council      1 2022-11-23
# i 3 more variables: `Session Type` <chr>, `Session Start-End Time` <chr>,
#   Present <chr>
```


Example (continued)

If we want to see just the last 6 rows of our dataset, we can use the `tail()` function:

```
# A tibble: 6 × 10
  `_id` Term      `First Name` `Last Name` Committee      `MTG #` `Session Date`
  <dbl> <chr>      <chr>        <chr>        <chr>        <dbl> <date>
1 12132 2022-2026 Lily          Cheng        Board of Heal... 32 2026-05-04
2 12133 2022-2026 Rachel        Chernos Lin Board of Heal... 32 2026-05-04
3 12134 2022-2026 Chris         Moise        Board of Heal... 32 2026-05-04
4 12135 2022-2026 Anthony      Perruzza     Board of Heal... 32 2026-05-04
5 12136 2022-2026 Michael      Thompson     Board of Heal... 32 2026-05-04
6 12137 2022-2026 Jamaal       Myers        Toronto Acces... 17 2026-05-04
# i 3 more variables: `Session Type` <chr>, `Session Start-End Time` <chr>,
#   Present <chr>
```

Example (continued)

Let's say we are interested in examining the attendance records of City Councillor Dianne Saxe, who represents the University–Rosedale ward, which includes the University of Toronto

We can subset our dataset to only include the records associated with Councillor Saxe's attendance:

```
1 saxe_data <- data |>  
2   filter(`Last Name` == "Saxe")
```

Example (continued)

```
1 saxe_data <- data |>  
2   filter(`Last Name` == "Saxe")
```

- We start by assigning the Councillor Saxe data to a new object (called `saxe_data`), which is separate from our main dataset
- We then use the `filter` function from `tidyverse` to subset the dataset and only keep the records for Councillor Saxe
- `==` checks if the value on the left is equal to the value on the right, in this case, `Last Name == "Saxe"` is checking whether the value in the Last Name column is exactly equal to the string "Saxe"

Example (continued)

```
1 saxe_data |>
2   group_by(Present) |>
3   count() |>
4   ungroup() |>
5   mutate(percentage = round(n / sum(n) * 100, 2))
```

- We use the `group_by()` function from `tidyverse` to group the data by the `Present` column by all possible values (“Yes” and “No”)
- The `count()` function counts the number of rows (or observations) in each group created by `group_by()`. It creates a new column, called `n` that contains the count for each group
- The `ungroup()` function removes any grouping that was previously set by `group_by()`

Example (continued)

```
1 saxe_data |>  
2   group_by(Present) |>  
3   count() |>  
4   ungroup() |>  
5   mutate(percentage = round(n / sum(n) * 100, 2))
```

- The `mutate()` function adds a new column called `percentage` to the dataset
- `n / sum(n) * 100, 2` calculates the percentage of each group and rounds the result to 2 decimal places

Example (continued)

```
# A tibble: 2 × 3
  Present      n percentage
  <chr>    <int>      <dbl>
1 N         26       5.68
2 Y        432      94.3
```

- We can see that Councillor Saxe has a 94% attendance rate, which is near the top/middle of all Councillors and the Mayor
- This could suggest, at a very high level, that because Councillor Saxe is physically present at many meetings, that she is able to better represent the interests of her constituents
- We'll be talking more about all of these R functions in the coming weeks

Your turn

- Go to the class GitHub repository, <https://github.com/inessadeangelis/INF2167> and to the “Week 1” folder
- Download the lab questions called `week1_lab.qmd` that are in that folder
- You can work individually or in small groups of 2 or 3

Submission

- Please upload your .qmd file and PDF to Quercus under “Assignments” – > “In-class Lab 1”
- If you are not in the class Quercus, please email your .qmd and PDF to Inessa.DeAngelis@mail.utoronto.ca with the subject line “INF2167: In-class Lab 1 Submission”

For next week

- *Telling Stories*, Chapters 3 & 4
- De Angelis, I. (2026). Using GitHub Actions for Computational Communication Research. *SocArXiv*. osf.io/preprints/socarxiv/uqf6n_v1
- **Optional:** *R for Data Science*, Chapter 6
- And if you haven't read this week's readings, please read those too

References

- Alexander, R. (2023). *Telling Stories with Data: With Applications in R*. Chapman & Hall/CRC.
- Dovi, S. (2020). What's missing? A typology of political absence. *The Journal of Politics*, 82(2), 559–571. <https://doi.org/10.1086/706979>
- Gelfand, S. (2022). *opendatatoronto: Access the City of Toronto Open Data Portal*. <https://CRAN.R-project.org/package=opendatatoronto>
- Pitkin, H. F. (1967). *The Concept of Representation*. University of California Press.
- Timbers, T., Campbell, T., & Lee, M. (2022). *Data Science: A First Introduction*. Chapman & Hall/CRC.
- Wickham, H., Averick, M., Bryan, J., Chang, W., McGowan, L. D., François, R., Golemund, G., Hayes, A., Henry, L., Hester, J., Kuhn, M., Pedersen, T. L., Miller, E., Bache, S. M., Müller, K., Ooms, J., Robinson, D., Seidel, D. P., Spinu, V., ... Yutani, H. (2019). Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- Wickham, H., Çetinkaya-Rundel, M., & Golemund, G. (2023). *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data* (2nd ed.). O'Reilly Media.